

Date: February 16th, 2021
From: David G Tassoni
To: Dr. Kaputa
Subject: ESD II Lab 1 Tennis Ball Detection

Introduction

The purpose of this laboratory was to create a Python, PyQt, and OpenCV based tennis ball tracker that will detect blue and green tennis balls within an image. There were four requirements, as follows:

- Shall be able to load in various images via a file menu
- Shall be able to select whether to detect a green or blue tennis ball
- Shall be able to save and load various parameters used for tennis ball detection from a configuration file
- Shall display the X and Y values of the centroid of the detected tennis ball

The below analysis investigates the methods as to which the above requirements were met, as well as the overall functionality of the Python program.

Analysis

The user will first begin by running the Python program. The user interface (UI) is shown below.

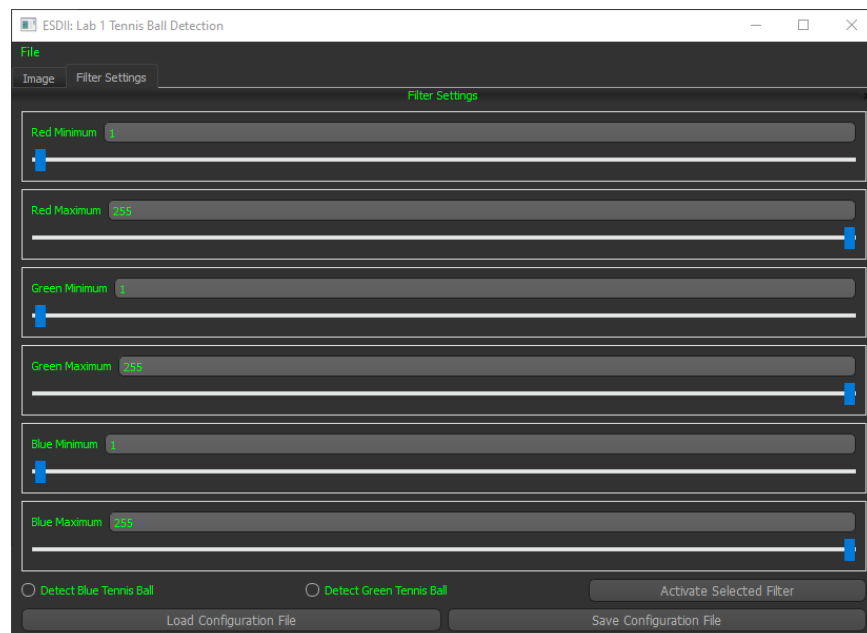


Figure 1 User Interface of the program

Upon running the program, the first step to analyzing is to open an image. Using the menu bar, simply select the File option, and the user has the option to "Open Image" or "Quit Application". The below images include the Python Code that creates the "Open Image" option in the File menu, and the code that runs when Open Image is selected.

```
#.Requirement.1: "shall.be.able.to.load.in.various.images.via.a.file.menu"
loadAction=QtGui.QAction(QtGui.QIcon('open.png'), '&Open Image', self)
loadAction.setShortcut('Ctrl+O')
loadAction.setStatusTip('Select Image to open')
loadAction.triggered.connect(self.openImage)
```

Figure 2 Code Snippet with Open Image File Option

```
def.openImage(self):
....fileName=QtGui.QFileDialog.getOpenFileName(self, "Select Image", "", "Image File (*.jpg)")
....if.not.fileName:
.....pass
....else:
.....globals.image_filepath=.fileName
```

Figure 3 The openImage function

Once the user opens an image, it is displayed in the Image section of the UI. He/she can open a configuration file, which sets the various parameters of minimum and maximum colors on the RGB scale. Once the configuration file is loaded, the user can choose to detect a blue or a green tennis ball and press the "Activate Selected Filter" button. The radio buttons for the two detection types meet the second requirement, and their associated Python code is shown below.

```
def.checkRadioButton(self, button):
#.Check which radio button is chosen, and load the associated RGB parameters
....if.button.text()=="Detect.Blue.Tennis.Ball":
.....if.button.isChecked():
.....with.open("parameters_blue.txt").as.f:
.....globals.rMin=float(f.readline().split("=")[1])
.....globals.rMax=float(f.readline().split("=")[1])
.....globals.gMin=float(f.readline().split("=")[1])
.....globals.gMax=float(f.readline().split("=")[1])
.....globals.bMin=float(f.readline().split("=")[1])
.....globals.bMax=float(f.readline().split("=")[1])
.....self.rMin.setValue(globals.rMin)
.....self.rMax.setValue(globals.rMax)
.....self.gMin.setValue(globals.gMin)
.....self.gMax.setValue(globals.gMax)
.....self.bMin.setValue(globals.bMin)
.....self.bMax.setValue(globals.bMax)
.....else:
.....pass

....if.button.text()=="Detect.Green.Tennis.Ball":
.....if.button.isChecked():
.....with.open("parameters_green.txt").as.f:
.....globals.rMin=float(f.readline().split("=")[1])
.....globals.rMax=float(f.readline().split("=")[1])
.....globals.gMin=float(f.readline().split("=")[1])
.....globals.gMax=float(f.readline().split("=")[1])
.....globals.bMin=float(f.readline().split("=")[1])
.....globals.bMax=float(f.readline().split("=")[1])
.....self.rMin.setValue(globals.rMin)
.....self.rMax.setValue(globals.rMax)
.....self.gMin.setValue(globals.gMin)
.....self.gMax.setValue(globals.gMax)
.....self.bMin.setValue(globals.bMin)
.....self.bMax.setValue(globals.bMax)
.....else:
.....pass
```

Figure 4 Radio Button Python code

The program reads the text on the button and loads the associated text file with the RGB values. The final step is to activate the filter, and this calls some methods in the faceTracker.py file. The program will draw a circle around the object it identified and mark the centroid in the circle. The coordinates are displayed in the upper-left hand corner of the image to provide more information.

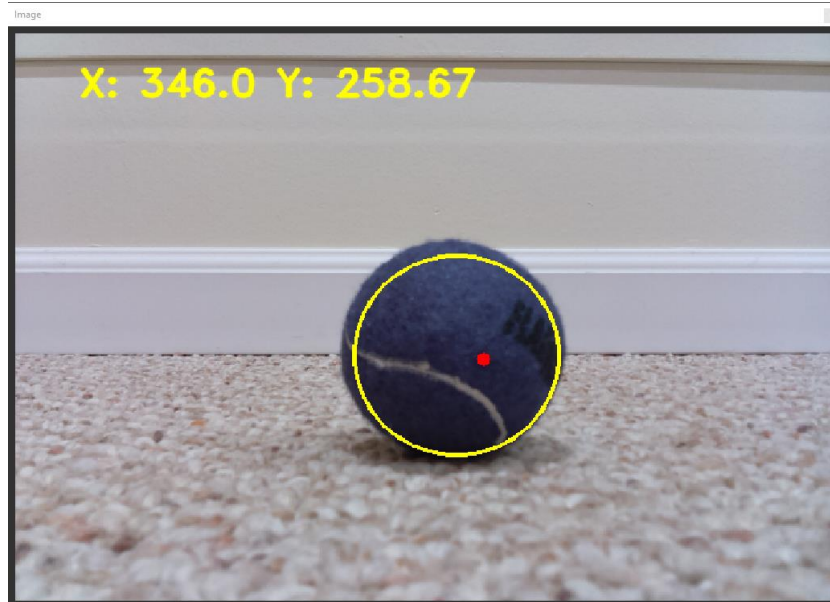


Figure 5 Detected Circle and Centroid

In this case, the program was set to detect a blue tennis ball, and the identified ball's circle and centroid are shown, with the coordinates printed in the top corner. The code that identifies the centroid and prints the coordinates is shown on the following page.

```

#Requirement 4: shall display the X and Y values of the centroid of the detected tennis ball
if len(contours) > 0:
    #find the largest contour in the mask and use it to compute the minimum enclosing circle
    maxContour = max(contours, key=cv2.contourArea)
    (x, y), radius = cv2.minEnclosingCircle(maxContour)
    maxContourMoments = cv2.moments(maxContour)
    center = (int(maxContourMoments["m10"] / maxContourMoments["m00"]), int(maxContourMoments["m01"] / maxContourMoments["m00"]))

    #move on when radius is bigger size
    if radius > 10:
        #draw the circle and centroid on the frame and update list of tracked points
        cv2.circle(image_clone, (int(x), int(y)), int(radius), (0, 255, 255), 2)
        cv2.circle(image_clone, center, 5, (0, 0, 255), -1)

        #font
        font = cv2.FONT_HERSHEY_DUPLEX

        #coordinates of text on image
        org = (50, 50)

        #fontScale
        fontScale = 1

        #color of text, I chose yellow
        color = (0, 255, 255)

        #thickness of text
        thickness = 2

        #Using cv2.putText() method
        image = cv2.putText(image_clone,
                            'X: ' + str(round(x, 2)) + ' Y: ' + str(round(y, 2)),
                            org,
                            font,
                            fontScale,
                            color,
                            thickness,
                            cv2.LINE_AA)
self.image = self.get_qimage(image_clone)
k = cv2.waitKey(5) #wait 5 milliseconds for any keyboard event
if k == 27:
    pass

```

Figure 6 Centroid identification and display

The program identifies contours and then identifies which is the largest. The OpenCV docs describe contours as “a curve joining all the continuous points (along the boundary), having same color or intensity. The contours are a useful tool for shape analysis and object detection and recognition.” Once the largest contour is identified, the program moves on to identify the moments, which is a term borrowed from physics. The CV2 function moments calculates all the moments up to the third order of a polygon or rasterized shape, and the middle of these moments is the centroid. The CV2 circle function is used to draw the circle and centroid, and the putText function is used to print the coordinates on the image.

Conclusion

The program is able to correctly filter images to identify and detect different colored tennis balls. Additional functionality of the program includes the ability to load images through a file menu, the option to detect a green or blue tennis ball, save and load various parameters to and from configuration files, and display the X and Y values of the centroid of the detected tennis ball.